

一, 子网(kind: Subnet) (这是 kube-OVN最大的特点, 不像flannel只有一个子网 Pod CIDR)

1. 子网可以定义自己的 CIDR, 网关, 网络策略和出网网关设置。
2. Namespace 可以绑定子网
3. 子网可以保留 IP 不被容器使用
 - Pod 支持固定 IP/MAC 地址, 通过 annotation定义。示例：
annotations:
 ovn.kubernetes.io/ip_address: 10.16.0.15
 ovn.kubernetes.io/mac_address: 00:00:00:53:6B:B6
 - Workload 类型 (Deployment/StatefulSet/DaemonSet/Job) 可以通过 IP Pool 注解固定 IP:
annotations:
 ovn.kubernetes.io/ip_pool: 10.16.0.15,10.16.0.16,10.16.0.17
 - StatefulSet 支持 IP 复用, StatefulSet 生命周期内, Pod 按名字复用 IP 信息。
 - 同时支持 EIP 绑定到 Pod
4. 子网支持 IPv6, 容器可获得 IPv6 地址
5. 分布式虚拟交换机/路由器, 数据平面无单点
6. 多集群网络互联: 属于不同集群的Pod 可以通过 Pod IP通信。(EKS可以用)
7. 多网卡支持:通过Multus-CNI, 但必须将 OVN 作为 Pod 的默认网络。
8. 用户可以自定义VPC (kind: Vpc)。不同 VPC 之间网络独立, 可以自由配置VPC内的IP 地址段、子网, 支持为自定义 VPC 配置路由。

二, 可设置子网出网方式, 通过修改 Subnet CRD 实例的设置

1. 分布式网关: 每个机器作为当前机器上容器的出网网关, 开启 NAT 的情况下出网地址为容器所在机器地址。
2. 集中式网关: 每个子网可以指定集群中的某台机器作为出网网关, 开启 NAT 的情况下出网地址为网关机器地址。集中式网关同时也支持高可用。(安全, 方便审计和白名单设置)
3. 容器地址直接对外暴露, 关闭 NAT 控制, 容器 IP 直接对外, 需要在外部路由器上加一条静态路由将容器网段流量引入集群内任意一台机器。(这个与 kuryr 给Pod 分配和 VM 相等地位的 IP 不同)

三, 安全

1. 支持 K8s Network Policy, 使用 ovs 流表实现, 不经过 iptables
2. 支持子网级别的网络控制。(比 Network Policy 粒度稍粗, 但更方便设置)
3. 支持双向带宽 QoS 控制, 可以动态更改 Pod 出入网络带宽
 - 以给 Pod 添加注解实现, 可动态更改注解:
annotations:
 ovn.kubernetes.io/ingress_rate: "3"
 ovn.kubernetes.io/egress_rate: "1"
4. 启用 Port Security, 避免 IP/MAC Spoofing
5. 流量镜像, 可将所有容器网络间通信流量复制, 方便之后做审计和故障排查
 - 默认会在每台机器上创建一块 mirror0 的虚拟网卡, 镜像本机的所有容器流量
 - 用户可以通过 tcpdump -nni mirror0 观察流量信息, 也可以通过其他工具将流量从 mirror0 网卡导出做进一步分析

四, 运维

1. 控制平面高可用
2. 控制平面升级不影响数据平面
3. 可使用 kubectl 插件检测网络状态
4. 可与 Prometheus 和 Grafana 集成:
 - 可以将网络控制平面信息以及网络数据平面质量信息指标以 Prometheus 所支持的格式对外暴露。

五, 性能

1. 使用 OVS 流表实现 Service 转发, 降低 iptables 使用, 提高性能。相对于 iptables 对每一个 svc 都要配置 4 条规则, 使用流表在大量 svc 的场景下更具性能优势。
2. 性能测试: 在阿里云 ECS 上, 以 Host 网络作为测试基准, 用 iperf3 作为工具, 比较 Kube-OVN 在网络吞吐量上的损耗:

包大小	Kube-OVN网络	Host网络	带宽比
128KB	2.42Gbit/s	2.52Gbit/s	96%
64KB	2.42Gbit/s	2.52Gbit/s	96%
16KB	2.42Gbit/s	2.52Gbit/s	96%
4KB	2.42Gbit/s	2.52Gbit/s	96%
2KB	2.39Gbit/s	2.51Gbit/s	95%

kube-ovn vs. kuryr-kubernetes vs. flannel

功能	kube-ovn	kuryr-kubernetes	flannel
不同namespace 网络隔离 (subnet, VPC, gateway)	支持	支持	不支持
IP固定 (deploy, statefulset, ds)	支持	不支持	不支持
FIP 直接访问Pod	支持	支持	不支持
QoS	支持	支持	支持 (bandwidth 插件)

Network Policy	支持	支持	不支持
Traffic mirror	支持	不支持	不支持
网络模式	Overlay (Geneve) Underlay (Vlan)	依赖neutron 网络	Overlay(vxlan) 路由(host-gw)
监控 (Prometheus 集成)	支持 (控制面 + 数据面)	部分支持 (Pod 网卡监 控)	部分支持 (Pod 网卡监控)

性能	kube-ovn	kuryr-kubernetes	flannel
创建单个Pod(网卡) 时间	1s (未设置访问规则)	10s-30s (瓶颈在neutron-ovs-agent + neutron-server)	1s
并发创建 20个Pods	1s	3mins-5mins	1s
单节点支持Pods数	110	110	110

其他	kube-ovn	kuryr-kubernetes	flannel
高可用	支持	支持	单节点插件，无中心 控制器
稳定性	未知	一般	稳定(单节点插件)
社区活跃度/成熟度	较好	较差	成熟度很高

由上面总结对比可见: kube-ovn 提供了很丰富的容器网络功能(租户隔离，访问控制)且具有较好的执行速度

引入kube-ovn 需求、可行性、利弊分析:

- EOS 控制平面: 控制平面Pods暂不需要租户隔离及访问控制需求, 性能/稳定性/成熟度三个方面考虑 flannel 都优于kube-ovn
- 安全容器: 因为依赖neutron-ovs-agent + neutron-server+neutron-lb, 安全容器网络方面有两个问题:
 1. 网卡创建速度慢
 2. 依赖网络(包括前端页面)云产品

安全容器使用ovn有两种方案:

- 通过kuryr对接neutron使用ovn: 可以优化网卡创建慢的问题, 依然依赖网络云产品
- 安全容器使用kube-ovn作为CNI(替换掉kuryr)

kuryr替换为kube-ovn好处:

1. 创建pods速度很快;
2. 不依赖网络云产品;
3. 社区支持更好;

kuryr替换为kube-ovn的弊端: 因为kube-ovn和neutron-ovn是两个独立ovn 北向入口, 安全容器和虚拟机网络默认是隔离的, 需要额外工作将两种网络做路由联通

对比上面两种方案, 建议优先测试通过kuryr对接neutron使用ovn, 测试创建网卡速度

- EKS集群: 当前默认CNI是flannel 网络功能过于简单, 对于更多网络功能的需求可以考虑调研对比kube-ovn及cilium/calico等其他成熟CNI插件
- kubevirt 可以使用 kube-ovn作为CNI: kubevirt使用pod封装qemu, 本身pod创建使用containerd, 所以使用的是cni插件完成ip, route, resolv的配置